

# Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Circuits and Automata</b>           | <b>1</b> |
| 1.1      | Languages                              | 1        |
| 1.2      | Deterministic Finite Automata          | 1        |
| 1.3      | Nondeterministic Finite Automata       | 2        |
| 1.3.1    | Stochastic Finite Automata             | 2        |
| 1.4      | Circuits                               | 2        |
| 1.4.1    | Function decomposition                 | 3        |
| 1.4.2    | Function evaluation by message passing | 6        |



# Chapter 1

## Circuits and Automata

**Abstract** We here present tensor network representation of standard models of computation. In particular, we derive TT representations of finite automata and represent generic logical circuits.

### 1.1 Languages

Languages over finite alphabets  $\Sigma$  are collection of strings in  $\Sigma^*$ . The symbols in  $\Sigma$  are enumerated by an index interpretation map  $I$ .

Collections  $\mathcal{U}$  of strings of length  $n$  are represented by a boolean tensor

$$\tau^{\mathcal{U},n} [O_{[n]}] = \sum_{v \in \mathcal{U}} \bigotimes_{l \in [n]} \epsilon_{I(v|_l)} [O_l] .$$

A language  $L$  is encoded by the infinite collection of tensors

$$\{\tau^{L,n} [O_{[n]}] : n \in \mathbb{N}\} ,$$

each representing the set of strings of length  $n$ .

We want to study how to decompose such tensors and relate tensor network schemes to standard models of computation.

### 1.2 Deterministic Finite Automata

Literature:

- [?] Basic Representation by TT/MPS.
- [?] Usage of canonical formats (orthogonalization) to find quantum circuit representations.

Idea:

- Construct chain graphs decorated with basis encodings of state transition functions, as in [?, ?]
- Contract with basis encoding of halting states and get the boolean tensor indicating the accepted states of the automaton

**Definition 1.1 (Deterministic finite automaton (DFA), see Def 2 in [?])** A tuple of

- set of internal states  $Q$
- alphabet  $\Sigma$
- initial state  $r_0 \in Q$
- accepted states  $F \subset Q$

- transition function  $\delta : Q \otimes \Sigma \rightarrow Q \cup \{\emptyset\}$

is called a deterministic finite automaton (DFA).

We enumerate the set  $Q$  by a map  $I_Q$  with enumeration variable  $O_Q$ , and the alphabet  $\Sigma$  by a map  $I_\Sigma$  with an enumeration variable  $O_\Sigma$ . Both indicator variables will be copied for  $k \in [d]$  (respectively  $k \in [d+1]$ ) and the copies denoted by  $O_{\Sigma,k}$  and  $O_{Q,k}$ . The acceptance tensor of order  $d \in \mathbb{N}$  is the boolean tensor  $\tau [O_{\Sigma,[d]}]$  indicating whether the automaton accepts the input sequence of length at most  $d$ . Its coordinates are

$$\tau [O_{\Sigma,[d]}] = \begin{cases} 1 & \text{if The automaton accepts the input sequence } (I_\Sigma(O_k = o_k) : k \in [d]) \\ 0 & \text{else} \end{cases}.$$

**Lemma 1.1 (Tensor representation)** *Let  $(Q, \Sigma, \delta, r_0, F)$  be a deterministic finite automaton. Then the acceptance tensor of order  $d \in \mathbb{N}$  is the contraction*

$$\tau [O_{\Sigma,[d]}] = \left\langle \{\beta^\delta [O_{Q,k}, O_{\Sigma,k}, O_{\Sigma,k}] : k \in [d]\} \cup \{\epsilon_{I_Q^{-1}(r_0)} [O_{Q,0}], \epsilon_{I_Q^{-1}(F)} [O_{Q,d}]\} \right\rangle_{[O_{\Sigma,[d]}]}.$$

*Proof.* By basis calculus: Given the one-hot encoding of the input word and contracting with the tensor network (expect the basis encoding tensor of  $F$ ) leaving  $O_{Q,d}$  open, we get the one hot encoding of the final state of the automaton. Contracting with the one-hot encoding of  $F$  is then 1, if and only if the final state is in the accepted states.  $\square$

Note that the acceptance tensor of the complement DFA is prepared when replacing the basis encoding  $\epsilon_{I_Q^{-1}(F)} [O_{Q,d}]$  by the basis encoding  $\epsilon_{I_Q^{-1}(\bar{F})} [O_{Q,d}]$  of the complement of the accepting states.

### 1.2.1 Outlook

We can use this theory to:

- Criterion that a language is not regular: Unbounded matriciation ranks in the word tensors. For example, this shows that  $\{0^n 1^n : n \in \mathbb{N}\}$  is not regular.
- Proof that regular languages are closed under regular operations: Use parallel TT cores, modifications in the transition functions.

## 1.3 Nondeterministic Finite Automata

Instead of transition function we have in NFAs a transition relation. Instead of directed boolean TT cores, we thus modify to generic boolean TT cores.

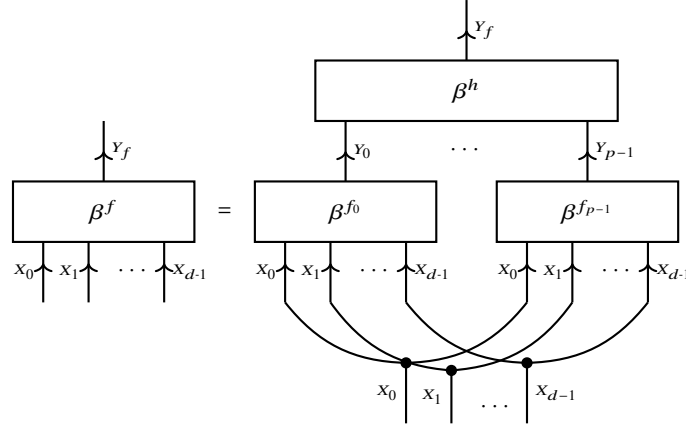
Usage:

- Proof that any NFA is a DFA: By using basis encodings of the cores understood as linear maps.

### 1.3.1 Stochastic Finite Automata

Ideas:

- The transition function is replaced by a cpd, which is encoded by a more general directed tensor and copied to all cores of a TT.
- The result is a Markov Chain, i.e. a Bayesian Network on a chain graph.
- The contraction above then gives the probability of acceptance for a given input word.
- One can add a distribution over the input words, and contract to get a joint distribution of input words with an acceptance variable.



**Fig. 1.1** Tensor network decomposition of the basis encoding of a function  $f$ , which is the composition of the functions  $f_0, \dots, f_{p-1}$  with a function  $h$ .

## 1.4 Circuits

The neural paradigm of artificial intelligence exploits the decomposition of functions into neurons, which are aligned in a directed acyclic graph. We show in this section how functions decomposable into neurons can be represented by tensor networks. To this end, we formalize discrete neural models by decomposition graphs and formally prove the corresponding decomposition of their basis encodings.

### 1.4.1 Function decomposition

As a main principle of tensor decompositions, we now show that basis encodings of composition functions are contractions of the basis encodings of their components.

**Lemma 1.2** *Let  $f[X_{[d]}]$  be a composition of a  $p$ -ary connective function  $h$  and functions  $f_\ell[X_{[d]}]$ , where  $\ell \in [p]$ , that is for  $x_{[d]} \in \times_{k \in [d]}[2]$  we have*

$$f(x_{[d]}) = h(f_0[x_{[d]}], \dots, f_{p-1}[x_{[d]}]) .$$

*Then, we have (see Figure 1.1)*

$$\beta^f[Y_f, X_{[d]}] = \langle \{\beta^h[Y_f, Y_{[p]}]\} \cup \{\beta^{f_\ell}[Y_\ell, X_{[d]}] : \ell \in [p]\} \rangle_{[Y_f, X_{[d]}]} .$$

*Proof.* For any  $x_{[d]} \in \times_{k \in [d]}[m_k]$  we have

$$\begin{aligned} & \langle \{\beta^h[Y_f, Y_{[p]}]\} \cup \{\beta^{f_\ell}[Y_\ell, X_{[d]}] : \ell \in [p]\} \rangle_{[Y_f, X_{[d]}=x_{[d]}]} \\ &= \langle \{\beta^h[Y_f, Y_{[p]}]\} \cup \{\beta^{f_\ell}[Y_\ell, X_{[d]}=x_{[d]}] : \ell \in [p]\} \rangle_{[Y_f]} \\ &= \langle \{\beta^h[Y_f, Y_{[p]}]\} \cup \{\epsilon_{f_\ell(x_{[d]})}[Y_\ell] : \ell \in [p]\} \rangle_{[Y_f]} \\ &= \epsilon_{f(x_{[d]})}[Y_f] \\ &= \beta^f[Y_f, X_{[d]}=x_{[d]}] . \end{aligned}$$

Thus, the tensors on both sides of the equation coincide in all slices to  $X_{[d]}$  and are thus equal.  $\square$

We now define a more generic decomposition of discrete functions.

**Definition 1.2** A *circuit* is a directed acyclic hypergraph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  such that the following holds.

- Each node  $v \in \mathcal{V}$  is decorated by a set  $\mathcal{U}^v$  of finite cardinality  $m_v$ , a variable  $X_v$ , and an index interpretation function

$$I_v : [m_v] \rightarrow \mathcal{U}^v.$$

- Each directed hyperedge  $(e^{\text{in}}, e^{\text{out}})$  has at least one outgoing node, that is  $e^{\text{out}} \neq \emptyset$ , and is decorated by an activation function

$$g^e : \bigtimes_{v \in e^{\text{in}}} \mathcal{U}^v \rightarrow \bigtimes_{v \in e^{\text{out}}} \mathcal{U}^v.$$

- Each node  $v \in \mathcal{V}$  appears at most once as an outgoing node.
- The nodes not appearing as an outgoing node are enumerated by  $v_{[d]}^{\text{in}}$ . We abbreviate the corresponding variables by  $X_{v_{[d]}^{\text{in}}} = X_{[d]}$ .
- The nodes not appearing as an incoming node are enumerated by  $v_{[p]}^{\text{out}}$ . We abbreviate the variables by  $X_{v_{[p]}^{\text{out}}} = Y_{[p]}$ .

We assign for each  $k \in [d]$  restriction functions

$$q_{v_k^{\text{in}}} : \bigtimes_{\tilde{k} \in [d]} \mathcal{U}^{\tilde{k}} \rightarrow \mathcal{U}^{v_k^{\text{in}}} \quad , \quad x_{[d]}|_k = x_k$$

to the nodes  $v_{[d]}^{\text{in}}$  and recursively assign to each further node  $v$  a node function

$$q_v : \bigtimes_{k \in [d]} \mathcal{U}^{v_k^{\text{in}}} \rightarrow \mathcal{U}^v \quad , \quad q_v(x_{[d]}) = g^{e_v} \left( \bigtimes_{\tilde{v} \in e^{\text{in}}} q_{\tilde{v}}(x_{[d]}) \right) |_v,$$

where  $e_v$  is to each  $v \in e^{\text{in}}$  the unique hyperedge with outgoing nodes  $\{v\}$ . We then call the function

$$q_{\mathcal{G}} : \bigtimes_{k \in [d]} \mathcal{U}^{v_k^{\text{in}}} \rightarrow \bigtimes_{\ell \in [p]} \mathcal{U}^{v_{\ell}^{\text{out}}} \quad , \quad q_{\mathcal{G}} = \bigtimes_{\ell \in [p]} q_{v_{\ell}^{\text{out}}}$$

the composition function to the circuit  $\mathcal{G}$ .

The neural paradigm in AI can be modeled by the existence of circuits for functions on large sets. We now show how circuits enable the sparse representation of composition functions by tensor networks.

**Theorem 1.1** For any circuit  $\mathcal{G}$  with composition formula  $q_{\mathcal{G}}$ , we have

$$\beta^{q_{\mathcal{G}}} [Y_{[p]}, X_{[d]}] = \left\langle \{ \beta^{g^e} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] : e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \} \right\rangle_{[Y_{[p]}, X_{[d]}]}.$$

*Proof.* We show by induction over the number of edges in  $\mathcal{G}$  that, for any  $x_{[d]} \in \bigtimes_{k \in [d]} [m_k]$  and  $v \in \mathcal{V}$ , we have

$$\left\langle \{ \beta^{g^e} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] : e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \} \right\rangle_{[X_{\mathcal{V} \setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]}]} = \bigotimes_{v \in \mathcal{V} \setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})} [X_v]. \quad (1.1)$$

$|\mathcal{E}| = 1$ : If there is a single edge  $e = (e^{\text{in}}, e^{\text{out}})$  in  $\mathcal{E}$ , we have  $X_{[d]} = X_{e^{\text{in}}}$  and  $\mathcal{V} \setminus v_{[d]}^{\text{in}} = e^{\text{out}}$ . In this case (1.1) holds since

$$\beta^{g^e} [X_{e^{\text{out}}}, X_{e^{\text{in}}} = x_{e^{\text{in}}}] = \bigotimes_{v \in \mathcal{V} \setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})} [X_v].$$

$(|\mathcal{E}| = n) \Rightarrow (|\mathcal{E}| = n + 1)$ : Let us now assume that (1.1) holds for all graphs with  $|\mathcal{E}| \leq n$  and let  $\mathcal{G}$  be a graph with  $|\mathcal{E}| = n + 1$ . We find an edge  $\tilde{e} = (e^{\text{in}}, e^{\text{out}})$  such that all nodes in  $e^{\text{in}}$  are only appearing as outgoing nodes in other edges.

$$\begin{aligned}
& \langle \{ \beta^{s^e} [X_e] : e \in \mathcal{E} \} \rangle_{\left[ X_{\mathcal{V} \setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]} \right]} \\
&= \left\langle \beta^{\tilde{e}} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] , \langle \{ \beta^{s^e} [X_e] : e \in \mathcal{E} \setminus \{\tilde{e}\} \} \rangle_{\left[ X_{\mathcal{V} \setminus \{v_{[d]}^{\text{in}} \cup e^{\text{out}}\}}, X_{[d]} = x_{[d]} \right]} \right\rangle_{\left[ X_{\mathcal{V} \setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]} \right]} \\
&= \left\langle \beta^{\tilde{e}} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] , \bigotimes_{v \in \mathcal{V} \setminus \{v_{[d]}^{\text{in}} \cup e^{\text{out}}\}} \epsilon_{q_v(x_{[d]})} [X_v] \right\rangle_{\left[ X_{\mathcal{V} \setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]} \right]} \\
&= \bigotimes_{v \in \mathcal{V} \setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})} [X_v] .
\end{aligned}$$

Here, in the second equation we used the induction hypothesis on the subgraph  $(\mathcal{V}, \mathcal{E} \setminus \{\tilde{e}\})$  with  $|\mathcal{E} \setminus \{\tilde{e}\}| = n$  and a generic contraction property of basis encodings in the third equation.

Thus, (1.1) holds always and we have for any  $x_{[d]} \in \times_{k \in [d]} [m_k]$  that

$$\begin{aligned}
\langle \{ \beta^{s^e} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] : e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \} \rangle_{[Y_{[p]}, X_{[d]} = x_{[d]}]} &= \left\langle \bigotimes_{v \in \mathcal{V} \setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})} [X_v] \right\rangle_{[Y_{[p]}, X_{[d]}]} \\
&= \bigotimes_{v \in v_{[p]}^{\text{out}}} \epsilon_{q_v(x_{[d]})} [X_v] \\
&= \epsilon_{q_{\mathcal{G}}} [Y_{[p]}] \\
&= \beta^{q_{\mathcal{G}}} [Y_{[p]}, X_{[d]} = x_{[d]}] .
\end{aligned}$$

Keeping  $X_{[d]}$  open, the claim is established.  $\square$

When neurons have tunable parameters, we can discretize those by sets  $\mathcal{U}^k$  and understand them as additional input variables.

*Example 1.1 (Sum of integers in  $m$ -adic representation)* We develop a tensor network representation of integer summations on the set  $[m^d] = \{0, \dots, m^d - 1\}$ , where  $m, d \in \mathbb{N}$ ,

$$+ : [m^d] \times [m^d] \rightarrow [m^{d+1}] \quad , \quad +(i, \tilde{i}) = i + \tilde{i} ,$$

which have a  $m$ -adic representation of length  $d$ . We define an index interpretation map

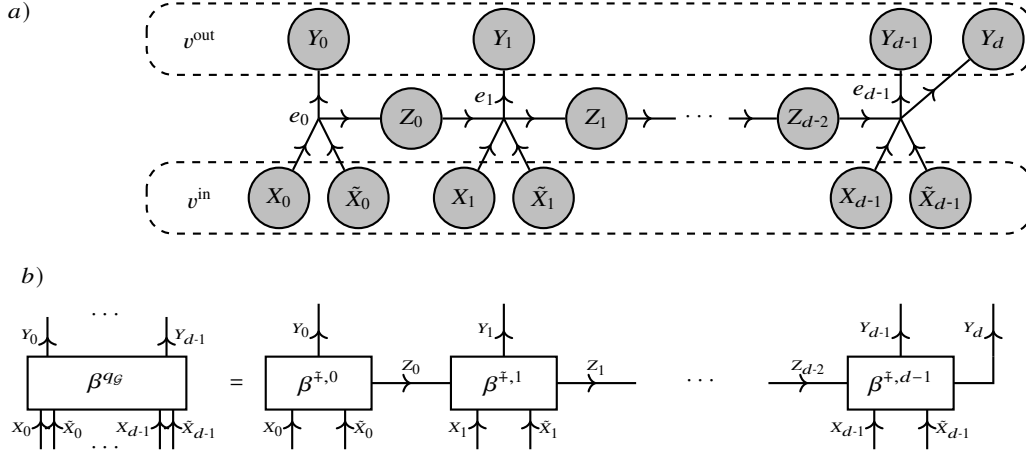
$$I : \bigtimes_{k \in [d]} [m] \rightarrow [m^d] \quad , \quad I(x_{[d]}) = \sum_{k \in [d]} x_k \cdot m^k ,$$

which enables the parameterization of  $[m^d]$  as the states of  $d$  categorical variables  $X_{[d]}$  of dimension  $m$ . We analogously represent a second set  $[m^d]$  by variables  $\tilde{X}_{[d]}$  and the set  $[m^{d+1}]$  of possible sums by  $Y_{[d+1]}$ . The basis encoding of the sum is then

$$\beta^+ [Y_{[d+1]}, X_{[d]}, Y_{[d]}] = \sum_{x_{[d]}, \tilde{x}_{[d]}} \epsilon_{I^{-1}(I(x_{[d]}) + I(\tilde{x}_{[d]}))} [Y_{[d+1]}] \otimes \epsilon_{x_{[d]}} [X_{[d]}] \otimes \epsilon_{\tilde{x}_{[d]}} [\tilde{X}_{[d]}] .$$

Note that the tensor space of  $\beta^+$  is of dimension  $m^{3 \cdot d+1}$  increasing exponentially in  $d$ . Feasible representation of this tensor for large  $d$  therefore requires tensor network decompositions, which we now provide based on a decomposition hypergraph. The targeted function to be decomposed is the representation of the integer sum by

$$+^m : \left( \bigtimes_{k \in [d]} [m] \right) \times \left( \bigtimes_{k \in [d]} [m] \right) \rightarrow \bigtimes_{k \in [d+1]} [m] \quad , \quad +^m(x_{[d]}, \tilde{x}_{[d]}) = I^{-1}(I(x_{[d]}) + I(\tilde{x}_{[d]})) .$$



**Fig. 1.2** Example of a decomposition hypergraph to the sum of integers (see Example 1.1). a) Hypergraph of directed edges  $e_k$  for  $k \in [d]$ , each decorated by an integer summation  $+$  preparing an index  $Y_k$  of the resulting sum. b) Corresponding tensor network decomposition of the basis encoded composition function, which is the sum of integers in  $m$ -adic representation.

We build a decomposition hypergraph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  (see Def. ??) consisting of  $4 \cdot d$  nodes (see Figure 1.2a). The nodes carry the  $(3 \cdot d + 1)$  variables  $X_{[d]}, Y_{[d]}, Y_{[d+1]}$  of dimension  $m$  constructed above and  $d - 1$  auxiliary variables  $Z_{[d-1]}$  of dimension 2 representing carry bits. The directed hyperedges of  $\mathcal{G}$  are

$$\mathcal{E} = \{(\{X_0, \tilde{X}_0\}, \{Y_0, Z_0\})\} \cup \{(\{Z_{k-1}, X_k, \tilde{X}_k\}, \{Y_k, Z_k\}) : k \in \{1, \dots, d-2\}\} \\ \cup \{(\{Z_{d-2}, X_{d-1}, \tilde{X}_{d-1}\}, \{Y_{d-1}, Y_d\})\}$$

and are decorated by local summation functions

$$\tilde{+} : [2] \times [m] \times [m] \rightarrow [m] \times [2] \quad , \quad \tilde{+}(z, x, \tilde{x}) = \left( (z + x + \tilde{x}) \bmod m, \left\lfloor \frac{z + x + \tilde{x}}{m} \right\rfloor \right).$$

Since to the first hyperedge we do not have a carry bit, the decorating function is the restriction of the first argument to 0.

It is known that the composition of the local summations  $\tilde{+}$  is the global summation  $+^m$  of integers in  $m$ -adic representation. Thus, the composition function  $q_{\mathcal{G}}$  is  $+^m$ . By Thm. 1.1 we have a decomposition of the basis encoding to  $q_{\mathcal{G}}$  (see Figure 1.2b) as

$$\beta^{+^m} [Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}] = \langle \{\beta^{\tilde{+},0} [Y_0, Z_0, X_0, \tilde{X}_0]\} \cup \\ \{\beta^{\tilde{+},k} [Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}] : k \in \{1, \dots, d-2\}\} \cup \\ \{\beta^{\tilde{+},d-2} [Y_{d-1}, Y_d, X_{d-1}, \tilde{X}_{d-1}, Z_{d-2}]\} \rangle [Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}].$$

### 1.4.2 Function evaluation by message passing

We are now concerned with an efficient inference algorithm based on tensor network contractions. To evaluate a function given as a tensor network decomposition of its basis encoding, the whole network has to be contracted. As this can be infeasible for large networks, a message passing algorithm based on local contractions can be applied, compare Algorithm ?? for a message passing algorithm for tensor networks on a on tree hypergraph.



**Algorithm 1** Directed Belief Propagation**Require:** Tensor network  $\tau^{\mathcal{G}}$  on a directed hypergraph  $\mathcal{G}$ **Ensure:** Messages  $\{\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$ 

Prepare directed message directions

$$\mathcal{E}^{\rightarrow} = \{((e_0^{\text{in}}, e_0^{\text{out}}), (e_1^{\text{in}}, e_1^{\text{out}})) : e_0^{\text{in}} \cap (e_1^{\text{in}}, e_1^{\text{out}}) = \emptyset, e_1^{\text{out}} \cap (e_0^{\text{in}}, e_0^{\text{out}}) = \emptyset, e_0^{\text{out}} \cap e_1^{\text{in}} \neq \emptyset\}$$

Initialize a message queue  $S = \{(e_2, e_0) : e_2 \text{ has empty incoming nodes}\}$ **while**  $S$  not empty **do**Pop a  $(e_0, e_1)$  pair from  $S$ 

Update the message

$$\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \langle \{\tau^{e_0} [X_{e_0}]\} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}, e_2 \neq e_1\} \rangle_{[X_{e_0 \cap e_1}]}$$

Update  $S$  by all messages  $(e_1, e_3)$  which have not yet been sent, if all messages  $(e_2, e_1)$  have been sent.**end while****return** Messages  $\{\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$ 

We apply the Directed Belief Propagation, Algorithm Algorithm 1, on a circuit, where we add hyperedges to each leaf node and assign one-hot encodings of input states. We then show that the messages are the one-hot encodings to the evaluations of the node functions.

**Theorem 1.2** *Let  $\mathcal{G}$  be a decomposition graph and let us add hyperedges containing single input nodes, which are decorated by one-hot encodings. Then the messages computed in Algorithm 1 are characterized by*

$$\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \bigotimes_{v \in e_0 \cap e_1} \epsilon_{q_v(x_{[d]})} [X_v] .$$

*Proof.* We show the theorem inductively over the messages computed in Algorithm 1. The first message is sent from an input edge  $\{[k]\}$  to an edge  $e$  of the decomposition graph and is by assumption the one-hot encoding of an input state  $\epsilon_{x_k} [X_k]$ .

We now assume that all previous messages satisfy the claimed equation at an arbitrary stage of the algorithm. The message computed in the while loop is then a contraction of one-hot encodings with basis encodings and

$$\begin{aligned} \chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] &= \langle \{\beta^{g^{e_0}} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] \} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}\} \rangle_{[X_{e_0 \cap e_1}]} \\ &= \langle \{\beta^{g^{e_0}} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] \} \cup \{\epsilon_{q_v(x_{[d]})} [X_v] : v \in e^{\text{in}}\} \rangle_{[X_{e_0 \cap e_1}]} \\ &= \bigotimes_{v \in e_0 \cap e_1} \epsilon_{q_v(x_{[d]})} [X_v] . \end{aligned}$$

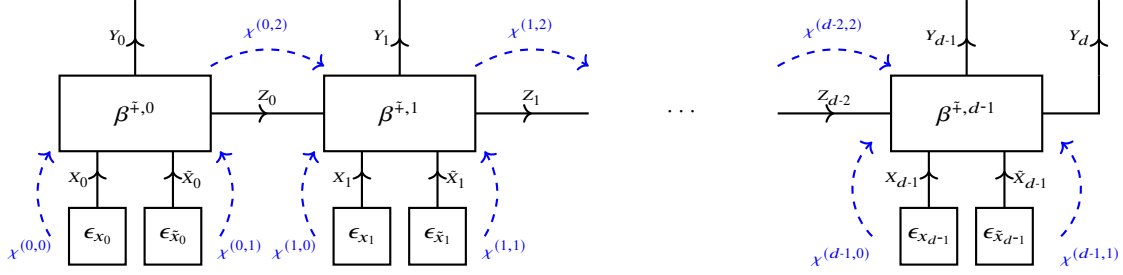
Thus, the new message is also the tensor product of the one-hot encodings of the evaluated node functions. By induction, the property is therefore true for all messages.  $\square$

We notice that we can interpret any directed acyclic hypergraph for which each node appears exactly once as an outgoing node and which is decorated by boolean and directed tensors  $\tau^{\mathcal{G}}$ . Edges with empty incoming sets are carrying one-hot encodings of input states and all further edges carry functions.

*Example 1.2 (Continuation of Example 1.1)* We now show how Algorithm 1 can be exploited to compute an efficient message passing algorithm for the digits of the  $m$ -adic sum. Given two numbers in  $m$ -adic representation by the tuples  $x_{[d]}$  and  $\tilde{x}_{[d]}$ , we add the hyperedges with empty incoming nodes and single outgoing node

$$\bigcup_{k \in [d]} \{(\emptyset, \{X_k\}), (\emptyset, \{\tilde{X}_k\})\}$$

to the hypergraph of Example 1.1 and decorate them by the digit one-hot encodings  $\epsilon_{x_k} [X_k]$  and  $\epsilon_{\tilde{x}_k} [\tilde{X}_k]$  (see Figure 1.3). We then apply the Directed Belief Propagation Algorithm 1. The initial messages queue then consists of the messages from the digit encoding. As sketched in Figure 1.3, to each digit there are three messages (with the exception



**Fig. 1.3** Computation of the integer sum in  $m$ -adic representation by the Directed Belief Propagation Algorithm 1 (see Example 1.2). The summands are represented by one-hot encodings of the digits  $x_{[d]}$  and  $\tilde{x}_{[d]}$ , from which the messages start. The  $k$ -th digit (for  $k \in \{0, \dots, d-1\}$ ) of the sum is computed based on the first messages of the epoch labeled by  $\chi^{(k,[2])}$ . The third message  $\chi^{(k,2)}$  in each epoch communicates the carry bit to the next digit summation core. In the last message epoch the digit  $d-1$  and  $d$  are computed based.

of the first being two), which can be scheduled in consecutive epochs  $\chi^{(k,[3])}$ . We then have by Thm. 1.2 for  $k \in [d-1]$  that

$$\langle \beta^{\mp,k} [Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}], \chi^{(k-1,2)} [Z_{k-1}], \chi^{(k,0)} [X_k], \chi^{(k,1)} [\tilde{X}_k] \rangle_{[Z_k]} = \epsilon_{z_k} [Z_k],$$

where  $z_k$  is the value of the  $k$ -th carry bit. The  $k$ -th digit of the sum  $y_k$  can further be obtained by the contraction

$$\langle \beta^{\mp,k} [Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}], \chi^{(k-1,2)} [Z_{k-1}], \chi^{(k,0)} [X_k], \chi^{(k,1)} [\tilde{X}_k] \rangle_{[Y_k]} = \epsilon_{y_k} [Y_k].$$

Note that the hypergraph representing this instance is a tree and by Thm. ?? also the message passing scheme of Algorithm ?? is guaranteed to produce the exact contractions. We can exploit this fact for example in the efficient computation of averages of the summation digits, when we have an elementary distribution of input digits. We emphasize that the directed belief propagation Algorithm ?? is exact even if the hypergraph fails to be a tree, provided that we have directed and boolean tensors..